

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 01 -

POR QUE CRIAR BIBLIOTECAS INTERNAS DE ML

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	5
SAIBA MAIS.....	7
MERCADO, CASES E TENDÊNCIAS	14
O QUE VOCÊ VIU NESTA AULA?	19
REFERÊNCIAS.....	20

LEANDRO

O QUE VEM POR AÍ?

Imagine uma equipe de ciência de dados repetindo o mesmo código de preparação de dados ou cálculo de métricas em vários projetos. Sem uma biblioteca interna de machine learning (ML), isso acontece o tempo todo. Cada novo modelo começa do zero, resultando em duplicações de código, implementações ligeiramente diferentes para a mesma função e muito retrabalho.

Por exemplo, dois times podem implementar uma mesma métrica de forma inconsistente, gerando resultados discrepantes. Esse cenário leva à dificuldade de manutenção – corrigir um bug exige alterar código em vários lugares – e dificulta o compartilhamento de boas práticas, já que não há um repositório único de conhecimento.

É como reinventar a roda a cada projeto, contrariando o princípio do DRY (Don't Repeat Yourself) que prega evitar repetições desnecessárias de lógica. Naturalmente, a qualidade do software sofre: maior chance de erros, inconsistências sutis entre modelos e tempo dos especialistas gasto em tarefas repetitivas em vez de inovação.

Felizmente, empresas de ponta mostram outro caminho. A solução está em desenvolver bibliotecas internas bem projetadas para centralizar essas funcionalidades comuns. Em vez de cada projeto copiar e colar código, a organização mantém um pacote reutilizável de componentes ML: funções de pré-processamento de dados, cálculos de métricas customizadas, conectores a bases de dados, pipelines padronizados, etc.

Os benefícios são enormes: reutilização de componentes (evitando retrabalho e garantindo que todos usem as mesmas funções testadas), aceleração de novos projetos (pois muitos blocos já estão prontos) e padronização de soluções de acordo com o contexto e as boas práticas da empresa.

Grandes players de tecnologia investiram pesado nisso – o Airbnb, por exemplo, criou internamente o Airflow para orquestração de pipelines, depois tornando-o aberto; hoje ele é amplamente usado para agendar fluxos de dados e ML. Já o Uber desenvolveu o Michelangelo, uma plataforma interna de ML que padroniza todo o ciclo de vida de modelos (do preparo de dados ao deploy) para centenas de equipes.

Essas histórias ilustram como bibliotecas e plataformas internas de ML resolvem problemas de fragmentação e escalam o impacto da IA: o código passa a ser fonte única da verdade, erros são corrigidos uma vez só e propagados a todos e cientistas de dados podem focar no que importa – criar modelos melhores, não reescrever utilidades. Nesta aula, você vai descobrir por que e como fazer isso, aprendendo com casos reais e preparando-se para aplicar na prática. Não se preocupe – você terá a ajuda das videoaulas para experimentar tudo isso! Vamos em frente?



HANDS ON

Chegou a hora da mão na massa! Nesta seção, vamos demonstrar a criação e o uso de uma biblioteca interna de ML na linguagem Python, que é uma das mais populares no ecossistema de ciência de dados. Prepare-se para ver na tela como encapsular funcionalidades reutilizáveis. Vamos construir, passo a passo, funções genéricas que poderiam pertencer a uma biblioteca interna – por exemplo, uma função de pré-processamento de dados e uma métrica customizada – e então utilizá-las em um miniprojeto.

A ideia é simular o cenário: primeiro desenvolvemos componentes uma vez, depois consumimos esses componentes em diferentes partes, garantindo consistência e economia de esforço. Fique atento(a) às explicações durante o código e pense em como algo parecido poderia estar em uma biblioteca compartilhada do seu time. Assista às videoaulas e acompanhe a execução do código a seguir, entendendo como centralizar lógica frequente pode facilitar muito a vida do(a) cientista de dados!

No exemplo, definimos duas funções que poderiam fazer parte de uma biblioteca interna: `custom_preprocess`, que realiza uma etapa de limpeza/padronização nos dados de entrada, e `custom_metric`, que calcula uma métrica de avaliação específica do negócio. Em seguida, vemos como um projeto usaria essas funções prontas da “biblioteca” em seu código principal. Note como isso evita duplicações, pois qualquer projeto futuro pode importar e usar as mesmas funções. Confira:

```
# Linguagem: Python
# Definição de funções que fariam parte da biblioteca interna de ML da
# empresa
def custom_preprocess(data):
    """
    Realiza padronizações nos dados:
    - Preenche valores ausentes com medianas
    - Converte categorias em one-hot encoding
    - Normaliza escalas de campos numéricos
    """
    # ... (implementação hipotética)
    processed_data = data # local onde seria implementada a lógica
```

```
    return processed_data

def custom_metric(y_true, y_pred):
    """
    Calcula uma métrica customizada de desempenho.
    Exemplo: acurácia ponderada por importância do caso (ilustrativo).
    """
    # ... (implementação hipotética)
    metric_value = sum(y_true == y_pred) / len(y_true) # local onde seria
    implementada a lógica de acurácia simples
    return metric_value

# Uso das funções da biblioteca interna em um projeto de ML
# (Suponha que custom_preprocess e custom_metric foram importados da
# biblioteca)
dados_processados = custom_preprocess(dados_brutos)
predicoes = modelo.predict(dados_processados)
resultado = custom_metric(labels_reais, predicoes)
print(f"Resultado da métrica customizada: {resultado:.2f}")
```

Código-fonte 1 – Exemplo de funções de pré-processamento e métrica personalizadas em uma biblioteca interna de ML

Fonte: Elaborado pelo autor (2025)

Como vimos, com essas funções definidas uma vez, qualquer projeto pode reaproveitá-las. Nos vídeos do hands on, você verá a execução e poderá testar variações. Experimente você também adaptar essas ideias ao seu contexto: que partes do seu workflow de ML poderiam virar funções reutilizáveis? Mantenha esse questionamento em mente durante a prática. Após pôr a “mão na massa” no código, vamos nos aprofundar na teoria e em detalhes importantes. Continue conosco na próxima seção e lembre-se de pausar e rever as videoaulas sempre que precisar, combinado?

SAIBA MAIS

Vamos agora mergulhar fundo nos conceitos e fundamentos por trás das bibliotecas internas de ML, entendendo por que elas são tão importantes e como implementá-las com excelência. Nesta seção, iremos explorar os aspectos técnicos e estratégicos: desde princípios de engenharia de software (como controle de versão, design de APIs, gerenciamento de recursos) até estudos de casos de sucesso e lições aprendidas. Prepare-se para uma abordagem densa e informativa, conectando teoria com a prática de ciência de dados.

Por que bibliotecas internas? Em projetos de ML, é comum observar uma proporção significativa de esforço dedicada a tarefas repetitivas ou “infraestrutura invisível”. Um artigo de Rodrigo Oliveira (2025) destaca os riscos da repetição de código: manutenção complicada, aumento da complexidade e inconsistência de comportamento. Sem bibliotecas internas, cada squad recria soluções similares, o que encarece a evolução do software e atrasa entregas.

Em contraste, reutilizar código via bibliotecas compartilhadas promove eficiência e qualidade. Componentes reutilizáveis – quando bem projetados – já vêm testados e validados, reduzindo a chance de bugs e divergências. Além disso, encoraja boas práticas: se todos usam a mesma função de carregamento de dados ou o mesmo cálculo de F1-score, por exemplo, todos passam a seguir o mesmo padrão, fortalecendo a governança de código.

Essa padronização também simplifica a colaboração entre times. Pense em uma pipeline de feature engineering: sem um pacote comum, duas equipes podem criar independentemente funções para calcular a mesma feature, duplicando trabalho; com uma biblioteca interna, elas convergem para uma única implementação reaproveitável, economizando tempo e garantindo que os modelos usem exatamente os mesmos dados derivados.

Um benefício notável é o ganho de velocidade na iniciação de novos projetos. Com uma boa biblioteca interna de ML, um novo projeto não começa de uma folha em branco – ele já tem à disposição “blocos de montar” prontos para tarefas corriqueiras (conexão a bancos de dados corporativos, pré-processadores padrão, funções de visualização etc.). Isso encurta o tempo de desenvolvimento e libera a equipe para

focar nas particularidades do problema. Na prática, times que adotam essa abordagem relatam economia de tempo e esforço. No Uber, antes do Michelangelo, estima-se que os desenvolvedores e desenvolvedoras gastavam 70% de seu tempo construindo e corrigindo pipelines e infraestrutura de ML para cada novo caso, sobrando apenas 20% do tempo para efetivamente treinar modelos.

Com a padronização trazida pela plataforma interna, esse cenário mudou radicalmente – a maior parte do tempo passou a ser empregada em modelagem e experimentos, enquanto a infraestrutura tornou-se transparente e reutilizável. Em termos de ROI (Return on Investment), investir em bibliotecas internas pode parecer custoso no início (exige engenheiros dedicados, desenho de arquitetura etc.), mas retorna multiplicado a longo prazo: cada projeto futuro concluído mais rápido e com menos erros reflete economia de dinheiro e ganho de competitividade.

Vale ressaltar que a qualidade do código tende a subir com o uso disseminado de uma biblioteca interna. Como ela concentra funcionalidades críticas utilizadas por muitos indivíduos, costuma receber maior atenção em testes automatizados, revisão de código e otimizações de desempenho. Desta forma, sai ganhando também a confiabilidade e o desempenho dos sistemas de ML da empresa. Uma função de uma biblioteca interna frequentemente envolve algoritmos e estruturas de dados otimizados – podendo até envolver rotinas de baixo nível para aproveitar CPU/GPU de forma mais eficiente.

Por exemplo, se várias equipes precisam de uma função de distância entre vetores para medidas de similaridade, ter essa função implementada de forma vetorizada em NumPy/C++ dentro de uma biblioteca interna garante que todos usufruam da versão mais rápida e escalável possível, em vez de cada um criar uma versão potencialmente menos eficiente. Isso toca em um ponto chave: controle de recursos. Bibliotecas internas podem incorporar controles de uso de memória e paralelismo que desenvolvedores(as) individuais talvez não implementem em soluções ad-hoc. Uma mesma função pode ser escrita para tirar proveito de multithreading ou de GPUs e, assim que otimizada na biblioteca, beneficia todos os usuários. Em suma, padronizar componentes também padroniza (e melhora) o desempenho.

Outro aspecto teórico importante é o controle de versões e compatibilidade. Ao criar bibliotecas internas de ML, é preciso adotar estratégias sólidas de versionamento

semântico, de forma que melhorias contínuas possam ser distribuídas sem “quebrar” os projetos existentes. Isso requer disciplina de engenharia de software: ao evoluir a biblioteca (por exemplo, adicionando uma nova funcionalidade de pré-processamento ou corrigindo um bug em uma métrica), deve-se documentar mudanças, deprecitar funções antigas gradualmente e comunicar equipes.

Ferramentas de gerência de pacotes (como registries internos do pip/Conda, em se tratando de Python) podem ajudar a distribuir e atualizar a biblioteca internamente. Empresas maduras costumam até ter “campeões” ou comitês de arquitetura responsáveis por revisar contribuições à biblioteca interna, garantindo sua coesão e qualidade ao longo do tempo. Aqui entra o conceito de Inner Source: aplicar práticas de código aberto dentro da empresa. Isso significa estimular que diferentes desenvolvedores(as) colaborem na biblioteca interna, quase como um projeto open-source, com revisões, issues e melhorias contínuas.

Essa abordagem, quando bem conduzida, evita que a lib se torne um feudo ou acabe abandonada – pelo contrário, torna-a viva e em melhoria constante, atendendo às necessidades reais dos times. Uma biblioteca interna efetiva não é estática; ela cresce e se adapta conforme novos desafios aparecem (novos tipos de modelo, novas fontes de dados etc.) (MORRIS, 2023).

Boas práticas de implementação: ao desenvolver (e manter) sua biblioteca interna de ML, há alguns princípios técnicos e de design a observar. Primeiro, escopo claro e foco: evite criar uma “mega biblioteca” que tenta resolver tudo e fica pesada ou genérica demais. Em vez disso, identifique componentes transversais realmente úteis (por exemplo, funções de validação de dados, conversão de formatos, cálculos estatísticos usados nos projetos) e construa módulos coesos em torno deles. Isso facilita manter a biblioteca “em forma” e minimiza acoplamentos indesejados. Lembre-se de que modularidade é essencial: talvez tenha módulos separados para “pré-processamento”, “engenharia de features”, “métricas” etc., para que consumidores importem apenas o necessário.

Segundo, invista em documentação e exemplos: uma biblioteca interna só será amplamente adotada se for bem compreendida. Providencie tutoriais internos, notebooks de exemplo, guias de referência – enfim, material que educa colegas desenvolvedores(as) a usá-la corretamente. Ter uma espécie de “cookbook”

mostrando como executar tarefas comuns com a biblioteca estimula sua utilização e diminui a curva de aprendizado, mitigando quaisquer resistências iniciais.

Terceiro, testes automatizados abrangentes: cada função ou classe da biblioteca deve vir acompanhada de testes unitários e, se possível, testes de integração. Isso não apenas garante a correta funcionalidade, mas dá confiança para evoluir o código sem medo. Se alguém sugere melhorar o algoritmo de pré-processamento X, os testes asseguram que o comportamento esperado (input -> output) permaneça correto após a mudança.

Quarto, pense em controle de memória e performance: bibliotecas de ML frequentemente lidam com grande volume de dados, então programação defensiva é bem-vinda (liberar memórias temporárias, evitar copiar estruturas desnecessariamente etc.). Uma boa prática é fornecer funções que trabalhem em streams de dados ou utilizem lazy evaluation quando possível, para não carregar tudo na memória de uma vez. Além disso, considere opções de paralelização: por exemplo, sua biblioteca interna pode fornecer uma versão paralela de alguma rotina custosa, que aproveita todos os núcleos da máquina ou distribui tarefas, se isso fizer sentido no ambiente da empresa.

Por fim, abordagem incremental: muitas empresas se perguntam “Quando faz sentido criar uma biblioteca interna? Devemos mesmo investir nisso agora?”. Uma recomendação comum (informalmente chamada “regra do terceiro uso”) é: quando você ver a mesma solução sendo implementada ao menos três vezes em projetos diferentes, é um sinal claro de que vale a pena abstrair aquilo em um componente compartilhado. Antes disso, talvez seja aceitável duplicar código uma ou duas vezes para testar ideias; mas à medida que a organização amadurece em ML, padrões repetitivos surgirão e não os aproveitar traz custo de oportunidade.

Também pese o cenário, pois há casos em que usar diretamente bibliotecas open-source é suficiente e preferível – afinal, não se deve reinventar a roda sem necessidade. A decisão de construir internamente versus adotar algo pronto deve considerar se as ferramentas abertas existentes atendem ao seu caso de uso. Por exemplo, se sua necessidade é um pipeline de ETL/ML orquestrado, talvez frameworks como Apache Airflow ou Kubeflow já resolvam grande parte do problema. Empresas como a Uber e Airbnb só desenvolveram suas soluções (Michelangelo,

Airflow) porque não encontravam, na época, algo que suprisse suas demandas específicas de escala e flexibilidade.

A abordagem sensata é usar open-source quando possível; criar biblioteca interna quando identificar um gap importante ou vantagem competitiva em ter algo sob medida. E mesmo ao criar, não isole do mundo: frequentemente a biblioteca interna vai se apoiar em bibliotecas externas consagradas (por exemplo, a interna pode “por baixo dos panos” usar pandas, scikit-learn, TensorFlow etc., apenas fornecendo conveniências ou integrações adaptadas ao contexto da empresa) (UBER ENGINEERING, 2017).

Para sintetizar os problemas vs. soluções que discutimos, observe a tabela a seguir, comparando desafios de não ter bibliotecas internas com os benefícios obtidos ao adotá-las:

DESAFIOS SEM BIBLIOTECA INTERNA DE ML	BENEFÍCIOS COM BIBLIOTECA INTERNA DE ML
Código duplicado em múltiplos projetos	Reutilização de código centralizado (evita retrabalho)
Implementações inconsistentes da mesma funcionalidade	Comportamento consistente em todos os projetos (uma única fonte)
Correção de bugs morosa e sujeita a esquecimentos	Manutenção única e simplificada (corrige na biblioteca, vale para todos)
Atraso no início de novos projetos (reinventar tudo)	Agilidade no kick-off: componentes prontos aceleram novas soluções
Ausência de padrões unificados e boas práticas	Padronização conforme contexto da empresa; difusão de boas práticas
Maior propensão a erros repetidos	Redução de erros: componentes comuns testados e comprovados

Tabela 1 – Desafios versus benefícios da adoção de bibliotecas internas de ML
Fonte: Elaborado pelo autor (2025)

Visivelmente, centralizar e compartilhar supera a fragmentação em praticamente todos os aspectos. Claro que precisamos estar cientes das armadilhas possíveis: se uma biblioteca interna não tiver dono ou comunidade ativa, pode ficar obsoleta; se for mal projetada, pode impor acoplamentos excessivos e inibir experimentações. Porém, seguindo as boas práticas mencionadas (foco, documentação, testes, versionamento, abertura à colaboração), as chances de sucesso aumentam muito.

Para concluir esta seção teórica, vale uma reflexão: engenharia de machine learning hoje vai muito além de treinar modelos. Envolve construir plataformas robustas que suportem o ciclo de vida inteiro, garantindo escalabilidade, reprodutibilidade e governança. Bibliotecas internas de ML são um dos alicerces dessa plataforma, ao lado de outras ferramentas (como automação de deploy, monitoramento de modelos em produção etc.). Elas endereçam diretamente o “déficit invisível” de projetos de IA – aquilo que não aparece nas demos, mas consome esforço gigante nos bastidores.

A famosa publicação do Google “Hidden Technical Debt in Machine Learning Systems” apontou que o código de ML propriamente dito é só uma pequena parte de um sistema completo de AI; ao redor dele há acúmulo de componentes de dados, pipelines, configurações, integrações. Organizar esse ecossistema de forma limpa e reutilizável é fundamental para que iniciativas de IA escalem dentro de uma organização. E, como você viu, criar bibliotecas internas de qualidade é uma estratégia poderosa para atingir esse objetivo, conferindo tanto ganhos imediatos (menos bug, menos retrabalho) quanto estruturais (base de código unificada, cultura de compartilhamento, facilidade para on-board de novos membros no time etc.).

Que tal você pensar, dentro do seu contexto ou empresa, quais componentes poderiam virar uma biblioteca interna? E se já existe alguma, como poderia ser melhorada? Afinal, aprender não é só absorver conteúdo, mas também aplicar criticamente. Não deixe de explorar as referências ao final para aprofundar ainda mais esses conceitos. E siga para a próxima seção, onde veremos o panorama de mercado, cases reais e tendências sobre esse assunto!

MERCADO, CASES E TENDÊNCIAS

A discussão sobre bibliotecas internas de ML está inserida em um contexto maior de MLOps e plataformas de IA empresariais, uma área em plena efervescência. Nos últimos anos, vimos uma forte movimentação de empresas investindo em suas próprias soluções internas ou adotando ferramentas open-source para gerenciar o ciclo de vida de machine learning. Vamos destacar aqui alguns casos marcantes – de sucessos a lições aprendidas – e tendências atuais que ilustram quando e como usar essa tecnologia de forma eficiente.

Airbnb e o caso do Airflow: um exemplo emblemático de sucesso é o Apache Airflow, mencionado anteriormente. Criado dentro do Airbnb em 2014 para automatizar e gerenciar workflows de dados, ele foi liberado como open source em 2015 e hoje é um pilar na comunidade de data science. Curiosamente, o Airflow passou por altos e baixos: alguns anos após o lançamento, o projeto enfrentou estagnação e queda de ritmo de atualizações. Porém, com o boom recente de IA e necessidades de MLOps, o Airflow ganhou uma “segunda vida”.

Em 2020, foi lançada a versão 2.0 e, desde então, a adoção explodiu – atualmente ultrapassa 35 milhões de downloads por mês. Isso mostra uma tendência: ferramentas internas bem-sucedidas podem transcender a empresa e virar padrões da indústria. A comunidade em torno do Airflow expandiu globalmente, incorporando melhorias para suportar cenários modernos, inclusive fluxos de ML e pipelines de treinamento de modelos. Para nós, interessa notar por que o Airflow foi criado: o Airbnb precisava coordenar tarefas complexas (por ex.: preparar dados, treinar modelos diariamente etc.) e não havia nada suficientemente flexível.

A solução interna se mostrou tão útil que outras empresas passaram a adotá-la, validando o esforço inicial. Hoje, o Airflow é praticamente sinônimo de orquestração de pipelines no ecossistema data/AI, ilustrando o valor de investir em plataformas internas quando se tem um problema comum e recorrente. Para saber mais, confira o [artigo](#) “Apache Airflow: From Stagnation to Millions of Downloads” – IEEE Spectrum (2025), que conta essa história e discute os planos de integrar mais capacidades de IA no Airflow.

Uber Michelangelo e plataformas de ML internas: outro caso de mercado significativo é o do Uber com sua plataforma Michelangelo. Desde 2017, o Uber compartilha insights de sua jornada de construir uma plataforma de ML unificada. O Michelangelo surgiu para democratizar o uso de ML internamente, padronizando desde a ingestão de dados até o monitoramento de modelos em produção. Ao longo dos anos, em vez de optar por soluções prontas, o Uber iterou nessa plataforma para atender a escala deles (milhares de modelos, exigência de latência baixíssima etc.).

O resultado? Em 2024, o Uber reportou que 100% dos casos de uso críticos de ML da empresa rodam sobre o Michelangelo, incluindo as novas aplicações de IA generativa (como bots de atendimento ao cliente). Ou seja, a plataforma interna conseguiu se manter relevante e evoluir para suportar até as tendências mais recentes de IA. Isso reforça uma tendência: grandes empresas tech preferem desenvolver soluções end-to-end internas para ML quando precisam de integração profunda com seus sistemas e dados. O Michelangelo inspirou outras corporações a pensarem de forma semelhante.

Hoje vemos, por exemplo, iniciativas no Facebook (como o FBLearn Flow), na Netflix (como o Metaflow) e em muitas outras, todas variando nos detalhes, mas perseguindo o mesmo objetivo – fornecer um ambiente integrado em que cientistas de dados possam mover modelos da ideia à produção rapidamente e com confiabilidade. Para quem quiser se aprofundar, há um [vídeo](#) excelente da MLOps Community chamado “ML Platform at Uber – Past, Present, and Future” (2024) com o Kai Wang, gerente de produto do Uber, explicando os aprendizados em 9 anos construindo o Michelangelo e os próximos passos da plataforma.

Feature Stores e o compartilhamento de features: uma tendência específica dentro de MLOps que ganhou tração é a de Feature Stores – essencialmente, repositórios internos de features para serem reutilizadas por modelos diferentes. Trata-se quase de uma “biblioteca interna de dados de entrada” para ML. A motivação é similar à das bibliotecas de código: sem um feature store, times recalculam as mesmas variáveis derivadas repetidamente; com ele, cada feature é computada uma vez, validada, documentada e colocada à disposição de todos.

Empresas pioneiras perceberam isso há alguns anos. Em 2019, a startup indonésia GO-JEK (junto com a Google) lançou como open source o Feast, um feature store concebido para unificar o armazenamento e acesso a features de ML. No

anúncio oficial, eles destacaram problemas comuns que o Feast resolve: features semelhantes sendo recriadas por times diferentes, definições que variam e causam inconsistência, dificuldade de servir dados atualizados de features para modelos em produção. O Feast centraliza essas definições e provê ferramentas para fácil consumo, tornando features em ativos reutilizáveis assim como código.

Hoje, feature stores se tornaram componentes-chave em muitas arquiteturas de ML enterprise – seja soluções open-source como o Feast ou proprietárias dentro de plataformas como o Michelangelo (o Uber implementou seu próprio feature store com milhares de features compartilhadas). A lição aqui é que compartilhar e padronizar não vale só para código, mas também para dados derivados, e isso tem sido uma tendência para melhorar a produtividade e a qualidade de modelos em larga escala (SELL.; PIENAAR, 2019).

Inner Source e colaboração interna: falando em tendências organizacionais, vale mencionar o movimento de Inner Source, que citamos brevemente. Grandes empresas, como Microsoft e IBM, têm relatado sucesso ao adotar práticas de comunidade open-source internamente para melhorar a reutilização de código. Em essência, elas tratam suas bibliotecas internas quase como projetos abertos, incentivando que engenheiros de diferentes produtos contribuam, revisem pull requests e usem as ferramentas uns dos outros. Essa cultura colaborativa quebra silos e aumenta a disseminação de soluções internas.

Estudos de caso (como o da IBM, “Inner Source na prática”) indicam que isso pode elevar significativamente a taxa de reuso de componentes e reduzir duplicações. A tendência, portanto, é cultural além de técnica: empresas estão percebendo que não basta ter a ferramenta, é preciso fomentar uma cultura de contribuição para mantê-la saudável. Uma biblioteca interna morre se ninguém além dos criadores a evolui; mas se dezenas de devs a encaram como “de todos nós”, suas chances de sucesso disparam. Portanto, muitas organizações em 2025 investem não só em infraestrutura de código, mas também em programas de incentivo para uso e melhoria das libs internas (hackathons internos, reconhecimento para contribuidores etc.) (MORRIS, 2023).

Abordagens híbridas e ferramentas third-party: por fim, é relevante notar a tendência de ferramentas combinadas. Nem toda empresa vai construir tudo do zero – o cenário atual oferece inúmeras soluções de MLOps prontas, e muitas vezes a

prática recomendada é integrar algumas delas. Por exemplo, uma empresa pode adotar o MLflow (open source da Databricks) para rastreamento de experimentos e gestão de modelos, usar o Airflow para orquestração e ainda assim desenvolver internamente uma pequena biblioteca com as funções utilitárias específicas do seu negócio. Essa mescla entre open source e código interno é muito comum.

Plataformas cloud também oferecem serviços gerenciados (como AWS SageMaker, Google Vertex AI, Azure ML Studio), que às vezes suplantam a necessidade de uma plataforma 100% interna, principalmente em empresas de menor porte. A tendência aqui é avaliar custo-benefício constantemente. Se a organização tem maturidade para manter uma biblioteca ou plataforma interna (como é necessário em gigantes de tecnologia), ótimo. Se não, há um ecossistema rico de produtos que podem ser “alugados” ou adaptados. Em resumo, a decisão de build or buy (construir ou comprar) no contexto de bibliotecas/plataformas de ML é um tópico quente. Vemos casos de sucesso em ambas as abordagens e muitas vezes uma estratégia híbrida vence. O importante é garantir os princípios que discutimos (reutilização, consistência, padronização), seja com ferramentas próprias ou de terceiros.

Em termos de futuro, podemos esperar ainda mais convergência entre desenvolvimento de software tradicional e pipelines de ML. Conceitos de engenharia de software (controle de versão, CI/CD, modularização) estão cada vez mais entranhados na rotina de equipes de ciência de dados. Novas ferramentas surgem a todo momento, mas a essência permanece: reduzir a barreira entre a concepção de um modelo e seu valor em produção. E bibliotecas internas de ML, com todo o conhecimento institucional encapsulado, continuarão sendo peça fundamental desse quebra-cabeça.

Para acompanhar as novidades, recomendamos estar atento a fontes confiáveis. Por exemplo, o podcast “MLOps Live” (Chip Huyen etc.) discute frequentemente casos de empresas implementando plataformas internas. Artigos em conferências como “Machine Learning Systems (MLSys)” também trazem estudos de ferramentas internas de empresas de ponta. Na seção de referências a seguir, você encontrará indicações interessantes – incluindo notícias recentes e publicações relevantes – que podem expandir sua visão sobre o tema.

- **Leitura recomendada 1:** [“The shared code fallacy: Why internal libraries can be an anti-pattern”](#) – Blog Ben Morris (2023). Um artigo de opinião que questiona a eficácia de bibliotecas internas em alguns casos, trazendo uma visão crítica e pontos de atenção (como acoplamento excessivo e desafios de manutenção). É um contraponto valioso para entendermos quando não usar uma biblioteca compartilhada.
- **Leitura recomendada 2:** [“Feast: Bridging ML Models and Data”](#) – Blog GO-JEK Tech (2020). Relato de como o feature store Feast foi criado e os problemas específicos que ele resolve em ambientes de ML. Bom para compreender a importância de reutilização de dados entre times de ciência de dados.
- **Vídeo:** [“Learnings from Building the ML Platform at Uber \(Michelangelo\)”](#) – MLOps Podcast (2024). Entrevista com engenheiros do Uber detalhando a evolução do Michelangelo, incluindo os desafios tecnológicos (como suportar modelos de deep learning, integrações com Spark etc.) e organizacionais ao escalar a plataforma. Disponível no YouTube.
- **Artigo de notícias:** [“Apache Airflow: from stagnation to millions of downloads”](#) – IEEE Spectrum (2025). Explora a história do Airflow, como a comunidade open source o revitalizou e porque ele se tornou fundamental em projetos de IA e Big Data recentes. Mostra também a importância da comunidade em torno de ferramentas internas que viram abertas.

Com esses exemplos e recursos, fica claro que o tema “bibliotecas internas de ML” evolui continuamente, impulsionado tanto por necessidades práticas quanto por uma cultura de colaboração na engenharia de dados e IA. A tendência é irreversível: compartilhar conhecimento via código (seja interno ou aberto) é cada vez mais parte do DNA de times de alta performance em ciência de dados.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos em detalhes por que criar bibliotecas internas de ML pode transformar a forma como times de ciência de dados trabalham. Iniciamos refletindo sobre os problemas da falta de padronização – código duplicado, inconsistências, dificuldade de manutenção – e entendemos os benefícios que uma biblioteca interna bem projetada traz, como reutilização de componentes, aceleração de projetos e padronização conforme o contexto da empresa.

Vimos um exemplo prático no hands on com Python, solidificando a ideia de encapsular funções úteis para uso compartilhado. Aprofundamo-nos em conceitos técnicos e boas práticas, tocando em princípios de engenharia de software (DRY, modularidade, testes, versionamento) e considerações de desempenho (uso de CPU/GPU, distribuição de processamento), sempre relacionando com a implementação de projetos de ciência de dados no mundo real.

Analizamos também cases de mercado: entendemos como empresas líderes como Airbnb e Uber criaram suas plataformas internas (Airflow, Michelangelo) e como tendências como Feature Stores e Inner Source estão moldando o futuro do desenvolvimento de ML. Com isso, você consolidou conhecimentos sobre quando e como investir em bibliotecas internas, equilibrando vantagens e cuidados, e viu exemplos concretos de sucesso (e alguns alertas) da indústria.

Em resumo, você aprendeu a importância de não reinventar a roda em projetos de IA e como bibliotecas internas atuam como catalisadores de eficiência, colaboratividade e qualidade de código. Este conteúdo lhe dá base para aplicar esses conceitos em seu ambiente profissional: identificar oportunidades de reuso, defender a implementação de uma biblioteca interna quando fizer sentido e adotar as melhores práticas para mantê-la sustentável.

Lembre-se: sempre que sentir necessidade, revise os pontos desta aula – o material está à sua disposição para consulta. E continue acompanhando as próximas aulas, em que expandiremos esses aprendizados em novos contextos de MLOps. Até lá, bons estudos e mãos à obra!

REFERÊNCIAS

CABALLAR, R. D. **Apache Airflow**: From Stagnation to Millions of Downloads. IEEE 2025. Disponível em: <https://spectrum.ieee.org/apache-airflow-3-programmatic-workflows>. Acesso em: 19 jan. 2026.

MORRIS, B. **The shared code fallacy**: Why internal libraries can be an anti-pattern. 2023. Disponível em: <https://www.ben-morris.com/the-shared-code-fallacy-why-internal-libraries-are-an-anti-pattern/>. Acesso em: 19 jan. 2026.

OLIVEIRA, R. **Repetição de Código – Entendendo os Riscos e Boas Práticas para Evitar Problemas**. 2025. Disponível em: <https://www.linkedin.com/pulse/repeti%C3%A7%C3%A3o-de-c%C3%B3digo-entendendo-os-riscos-e-boas-pr%C3%A1ticas-para-evitar-problemas-oliveira-8hlof>. Acesso em: 19 jan. 2026.

OPSLEVEL. **What is code reuse and why is it important?** 2025. Disponível em: <https://www.opslevel.com/resources/what-is-code-reuse-and-why-is-it-important>. Acesso em: 19 jan. 2026.

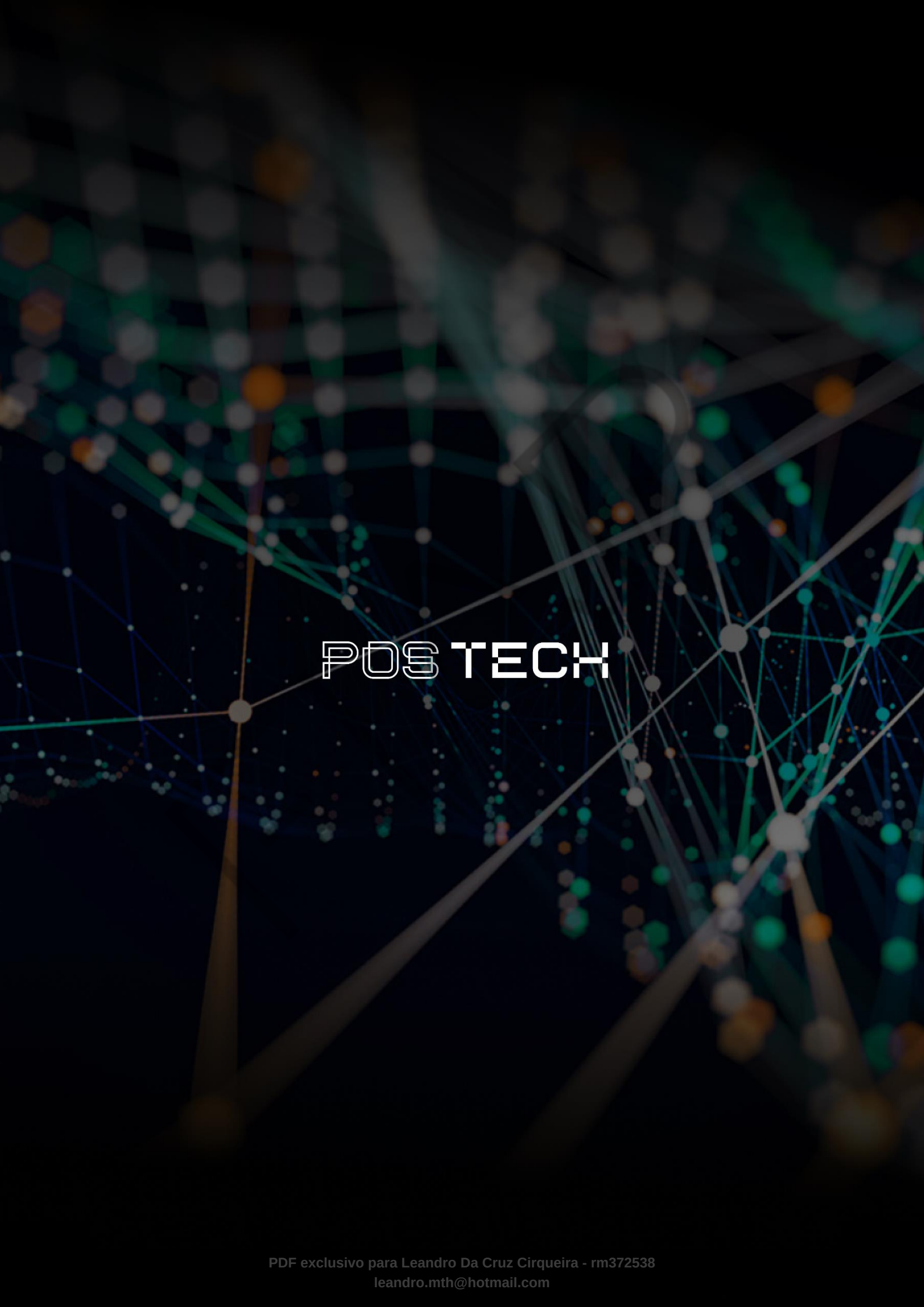
SELL, T.; PIENAAR, W. **Introducing Feast**: an open source feature store for machine learning. 2019. Disponível em: <https://cloud.google.com/blog/products/ai-machine-learning/introducing-feast-an-open-source-feature-store-for-machine-learning>. Acesso em: 19 jan. 2026.

UBER ENGINEERING. **Meet Michelangelo**: Uber's Machine Learning Platform. 2017. Disponível em: <https://www.uber.com/blog/michelangelo-machine-learning-platform/>. Acesso em: 19 jan. 2026.

PALAVRAS-CHAVE

Reutilização de código. Bibliotecas internas de ML. MLOps.

EMENDAS



POSTECH